

ForgeBOM — Implementation Plan (10 Phases, Local-First PWA)

Project summary

A local-first, offline PWA (Vite + Vue) for managing **raw materials, products, multi-level BOMs (including middle products)**, and **production calculators**. Product costs auto-recalculate from material prices, with **price history** and **BOM versioning**. Data stored locally (IndexedDB) with export/import backup.

Non-negotiables (from your requirements)

- Offline-first: app must fully run client-side; no auth/server for MVP.
- Local persistence: all user data stored locally.
- Unit conversions: **intra-dimensional only** (e.g., L↔ml, kg↔g). No mass↔volume.
- Quantities: support up to **5 decimals** (e.g., 0.00001) for BOM quantities.
- Currency: single currency (Toman), integer values (no decimals).
- BOMs: editable anytime; products can be components in other products.
- Two product types: **Middle** and **Final**.
- Costs: materials-only now; architecture must allow “full cost” later (labor/machine/overhead).
- Images: up to **3 medium images per product**, compressed ($\leq \sim 1\text{MB}$ each). Use thumbnails for lists.
- Scale target: up to $\sim 10\text{k}$ data rows per user (MVP).

Core concepts you should standardize early

1) Product “recipe basis” (yield) to handle ratios cleanly

Each product must have: - **Unit** (e.g., L, ml, kg, g, unit) - **Recipe yield quantity** in that unit (e.g., “This BOM produces **1 L** of Product Y”).

BOM line quantities are defined **relative to that yield**. - Example: Product Y yield = 1 L. BOM includes 800 ml water + 100 g A + ... - Another product X uses 333 ml of Product Y.

For calculations, you scale by: - $\text{scale} = \text{required_qty_in_product_unit} / \text{product_yield_qty}$ - Required raw materials = sum over BOM lines: $\text{line_qty} * \text{scale}$ (after unit conversion within dimension).

This avoids needing “sum of inputs equals output.” Inputs can be higher/lower; yield is the definition.

2) Multi-level BOM expansion

When a BOM line references a **product**, recursively expand until only raw materials remain. - Must support nested products (middle products). - Must prevent cycles ($A \rightarrow B \rightarrow A$).

3) Versioning model (audit + easy rollback)

- BOM edits create a **new BOM version** (immutable snapshot).
- Product points to `currentBomVersionId`.
- Keep a minimal changelog per version (optional but recommended).

4) Price history model

- Material has many price records: (`effectiveFrom`, `effectiveTo?`, `priceToman`).
 - "Current price" = record with no `effectiveTo`.
 - Future: cost "as of date" is possible with the same model.
-

Architecture blueprint (Clean Architecture in a Vue app)

Domain layer (pure)

- Entities/Value Objects:
 - `Material`, `MaterialPrice`
 - `Product`, `ProductImage`
 - `BomVersion`, `BomLine`
 - `Quantity` (value + unit + dimension + 5-decimal support)
- Domain services:
 - `UnitConverter`
 - `BomValidator` (cycle detection, unit compatibility)
 - `CostCalculator` (materials-only now; later add cost layers)
 - `RequirementsExpander` (expand nested BOMs)

Application layer (use cases)

- `CreateMaterial`, `UpdateMaterialPrice`, `ListMaterials`
- `CreateProduct`, `UpdateProduct`, `AttachProductImages`
- `CreateBomVersion`, `EditBom`, `SetCurrentBomVersion`
- `RecalculateCostsForImpactedProducts`
- `CalculateRequirementsForOrder`
- `ExportData`, `ImportData`, `BackupRestore`

Infrastructure layer

- IndexedDB repositories + schema migration
- Image compression pipeline (client-side)
- Optional: Web Worker for heavy recalcs

UI layer

- Views/pages: Materials, Products, BOM Editor, Calculator, Gallery, Settings/Backup
- State mgmt: Pinia (thin orchestration only)
- Composables: `useMaterials`, `useProducts`, `useBomEditor`, `useCosting`, `useBackup`

Data model (MVP)

Material

- `id` (stable, random/ULID)
- `name`
- `unit` (the default unit user registers with)
- `dimension` (mass/volume/count)
- `createdAt`, `updatedAt`

MaterialPrice

- `id`
- `materialId`
- `priceToman` (integer)
- `effectiveFrom`
- `effectiveTo?` (null = current)

Product

- `id`
- `type` (middle | final)
- `name`
- `unit` (product unit)
- `dimension` (volume/mass/count)
- `yieldQty` (recipe basis quantity in product unit)
- `description`
- `images[]` (up to 3)
- `currentBomVersionId?`
- `computedCostMaterialsOnly` (cached)
- `computedPrice` (optional: cost + margin)
- `createdAt`, `updatedAt`

BomVersion

- `id`
- `productId`
- `versionNumber`
- `createdAt`
- `notes?`

BomLine

- `id`
- `bomVersionId`
- `inputType` (material | product)

- `inputId`
- `qty` (decimal up to 5)
- `unit`
- `wastePct?` (optional, simple)
- `sortOrder`

Dependency index (for fast recalculation)

- `ProductDependency` :
- `productId` (the one whose BOM contains something)
- `dependsOnType` (material|product)
- `dependsOnId`

(Keep this updated whenever a BOM version becomes current.)

Costing & calculator rules (MVP)

Materials-only product cost

For a product P: - Expand BOM to raw materials (recursively). - For each raw material m: total required qty in m's unit × current price per unit. - Sum = `costMaterialsOnly`.

Handling waste/yield simply

- Product yield is already the "output definition."
- Add optional `wastePct` at BOM line level:
- $\text{effectiveQty} = \text{qty} * (1 + \text{wastePct})$

Performance / correctness constraints

- Detect cycles on BOM save.
 - If required product/component unit dimension mismatches, show validation error.
 - Use decimal math discipline (avoid floating rounding surprises by normalizing + fixed precision).
-

PWA & offline expectations

- Precache app shell (routes/assets) so the app launches offline.
 - Cache versioning: show "Update available" banner when a new SW is waiting.
 - Local DB migrations must be safe and forward-compatible.
-

10 Phases (each shippable)

Phase 1 — Foundations & project skeleton

Goal: a running offline PWA shell with navigation + architecture scaffolding. - Vite + Vue project - Routing + layout system - PWA setup (manifest, service worker) - Clean architecture folder boundaries - UI kit selection, typography, spacing rules - Basic “Settings” page with app version + storage usage

Deliverables - App installable on phone - Works offline after first load - Documented module boundaries + naming conventions

Definition of done - Lighthouse PWA checks mostly green - No business logic in components beyond display

Phase 2 — Local database & repositories

Goal: IndexedDB storage with migrations + repository API. - DB schema design + versioned migrations - Repositories for Material/Product/BOM/Images - Import/export format draft (JSON) - Error handling & recovery strategy

Deliverables - CRUD operations working in isolation - Simple demo UI to insert/read sample records

Definition of done - Migration from v1→v2 tested - Data survives reload/offline

Phase 3 — Units, quantities, and validation core

Goal: unit system + 5-decimal quantity support + validation helpers. - Define dimensions (mass/volume/count) - Define allowed units per dimension - Unit conversion table (intra-dimensional) - Quantity normalization + formatting rules - Validators: unit compatibility, nonnegative, max decimals, etc.

Deliverables - Unit conversion tested (ml↔L, g↔kg, etc.) - Shared quantity input component behavior spec

Definition of done - Conversions are deterministic + consistent in UI

Phase 4 — Materials module (CRUD + price history)

Goal: materials management that can scale and supports future pricing logic. - Create/update/delete material - Add “Update price” procedure: - closes previous price record (effectiveTo) - creates new current price record - List/search materials - Basic price history view

Deliverables - Materials screen usable end-to-end - Price update flow + history visible

Definition of done - Updating price never overwrites history

Phase 5 — Products module (CRUD + images)

Goal: define middle/final products with units, yield qty, and media. - Create/update product - Set product type (middle/final) - Set unit + yieldQty (recipe basis) - Add up to 3 images: - compress + store blob - generate thumbnail - Product list/search

Deliverables - Product detail page with images + description

Definition of done - Image lists load fast (thumbs)

Phase 6 — BOM editor + BOM versioning

Goal: fully editable BOM with version history. - BOM editor UI: - add lines (material/product) - qty + unit per line - reorder lines - wastePct optional - Saving creates new BOM version (immutable) - Set current BOM version - Validation: - cycle detection - dimension checks

Deliverables - “BOM Versions” list + revert switch

Definition of done - User can safely edit BOM anytime and recover older versions

Phase 7 — Cost engine + automatic recalculation

Goal: cached product costs that update when inputs change. - Implement recursive expansion to raw materials - Compute materials-only cost - Cache computed cost on product record - Dependency index maintenance - Trigger recalculation when: - material price updated - product BOM current version changes - nested product changes - Optional: Web Worker if recompute blocks UI

Deliverables - Product page shows computed cost breakdown - Material price change updates affected products' costs

Definition of done - Recalc is correct for nested BOMs - UI stays responsive for typical datasets

Phase 8 — Production calculator (requirements for N units)

Goal: calculate required raw materials for producing N of any product. - User picks product + enters target quantity (in product unit) - Expand recursively using yield scaling - Aggregate same materials into a single list - Output: - material name - required qty (formatted, 5 decimals) - unit - cost contribution - Optional: export calculator result to file

Deliverables - Calculator page works offline and is fast

Definition of done - Handles nested products correctly

Phase 9 — Final products gallery + filters + UX polish

Goal: a usable showcase/gallery for final products. - Cards with thumbnail, name, short description, final price/cost - Search + filters (tags/type/cost range) - Sorting (updated, name, price) - Batch actions (duplicate product, duplicate BOM template) - Templates: - Create BOM template from existing product - New product from template

Deliverables - Gallery feels like a real “product catalog”

Definition of done - Smooth scrolling + instant search on typical dataset

Phase 10 — Backup/Import, hardening, and release staging

Goal: make it safe to use and easy to maintain. - Export: - JSON export of all entities + blobs (or blobs as separate package) - include schema version - Import: - validate - merge strategy (MVP: overwrite or new workspace) - Storage health: - show usage - warn if nearing quota - Release UX: - update-available banner - changelog notes - Testing: - unit tests for converters/calculator/cycle detection - end-to-end smoke tests - Documentation: - user guide - developer guide

Deliverables - MVP release checklist completed

Definition of done - User can back up and restore reliably - Regressions caught by tests

AI-first implementation workflow (so nothing falls behind)

Deliverable-driven prompting

For each phase, write prompts that: - define the user story - define acceptance criteria - define edge cases (units, decimals, cycle detection) - define data model updates

Documentation habit

At the end of each phase: - Update “Data Model” section with schema changes - Add “Decisions” log (what you chose + why) - Add “Known limits” list - Add screenshots/gifs checklist

Phase checklist template

- [] User stories written

- [] UI flow sketch
 - [] Domain rules enumerated
 - [] DB schema updated + migration plan
 - [] Tests for the critical logic
 - [] Performance sanity check
 - [] Export/import compatibility check
-

Open items (next decisions you should make)

1. **BOM quantity semantics UI:** do you want BOM lines displayed “per yield” (e.g., per 1 L), and calculator input is arbitrary N? (recommended)
 2. **Rounding/display policy:** store 5 decimals, but how many to display by default in UI (2/3/5)?
 3. **Workspace concept:** do you need multiple factories/projects per device? (affects export/import and future multi-user)
-

Next step

Reply with your preference for: - (A) Single workspace MVP (simpler) - (B) Multiple workspaces from day 1 (more scalable)

...and how many decimals you want to display by default (storage stays 5).